1. **First Palindrome**

Program:

```
#include <stdio.h>
#include <string.h>

int isPalindrome(char str[])
{
   int i = 0;
   int j = strlen(str) - 1;

   while (i < j)
   {
      if (str[i] != str[j])
         return 0;
      i++;
      j--;
   }

   return 1;
}

int main()
{
   int n;

   printf("Enter number of words: ");
   scanf("%d", &n);

   char words[n][100];

   printf("Enter the words:\n");

   for (int i = 0; i < n; i++)
      scanf("%99s", words[i]);

   for (int i = 0; i < n; i++)
   {
      if (isPalindrome(words[i]))
      {
         printf("First palindrome: %s\n", words[i]);
         return 0;
      }
   }

   printf("No palindrome found\n");

   return 0;
}
```

Output:

```
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> & "C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\1.exe"
Enter number of words: 5
Enter the words:
abc
car
ada
racecar
cool
First palindrome: ada
```

2. **Common Elements**

Program:

```
#include <stdio.h>

int exists(int arr[], int n, int x)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == x)
            return 1;
    }
    return 0;
}

int main()
{
    int n, m;

    printf("Enter size of first array: ");
    scanf("%d", &n);

    int nums1[n];

    printf("Enter elements of first array:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &nums1[i]);

    printf("Enter size of second array: ");
    scanf("%d", &m);

    int nums2[m];

    printf("Enter elements of second array:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &nums2[i]);

    int answer1 = 0, answer2 = 0;

    for (int i = 0; i < n; i++)
```

```
    {
        if (exists(nums2, m, nums1[i]))
            answer1++;
    }

    for (int i = 0; i < m; i++)
    {
        if (exists(nums1, n, nums2[i]))
            answer2++;
    }

    printf("\nAnswer 1 = %d\n", answer1);
    printf("Answer 2 = %d\n", answer2);

    return 0;
}
```

Output:

```
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter size of first array: 3
Enter elements of first array:
2 3 2
Enter size of second array: 2
Enter elements of second array:
1 2

Answer 1 = 2
Answer 2 = 1
```

3. **Distinct Squares**

Program:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
```

```
    long long sum = 0;

    for (int i = 0; i < n; i++)
    {
        int unique[n];
        int count = 0;

        for (int j = i; j < n; j++)
        {
            int found = 0;

            for (int k = 0; k < count; k++)
            {
                if (unique[k] == nums[j])
                {
                    found = 1;
                    break;
                }
            }

            if (!found)
            {
                unique[count] = nums[j];
                count++;
            }

            sum += (long long)count * count;
        }
    }

    printf("\nSum = %lld", sum);

    return 0;
}
```

Output:

```
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> "C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\3.
exe"
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\3.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "3.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 3
Enter array elements:
1 2 1

Sum = 15
```

4. **Equal Pairs**

Program:

```
#include <stdio.h>
```

```
int main()
{
    int n, k;

    printf("Enter array size: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    printf("Enter k: ");
    scanf("%d", &k);

    int count = 0;

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (nums[i] == nums[j] && (i * j) % k == 0)
                count++;
        }
    }

    printf("\nNumber of valid pairs = %d", count);

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\4.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "4.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 7
Enter array elements:
3 1 2 2 2 1 3
Enter k: 2

Number of valid pairs = 4
```

5. **Maximum Element**

Program:

```
#include <stdio.h>
```

```
int main()
{
   int n;

   printf("Enter array size: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("Array is empty");
      return 0;
   }

   int arr[n];

   printf("Enter array elements:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   int max = arr[0];

   for (int i = 1; i < n; i++)
   {
      if (arr[i] > max)
         max = arr[i];
   }

   printf("\nMaximum Element = %d", max);

   return 0;
}
```

Output:

```
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> "C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\5.
exe"
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\5.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "5.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 5
Enter array elements:
1 2 3 4 5

Maximum Element = 5
```

6. **Sort Maximum**

Program:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right)
```

```
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int i = 0; i < n2; i++)
        R[i] = arr[mid + 1 + i];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);
```

```
    if (n == 0)
    {
        printf("List is empty");
        return 0;
    }

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("\nSorted Array:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nMaximum Element = %d", arr[n - 1]);

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\6.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "6.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 1
Enter array elements:
5

Sorted Array:
5
Maximum Element = 5
```

7. **Unique Elements**

Program:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);
```

```
    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int unique[n];
    int count = 0;

    for (int i = 0; i < n; i++)
    {
        int found = 0;

        for (int j = 0; j < count; j++)
        {
            if (arr[i] == unique[j])
            {
                found = 1;
                break;
            }
        }

        if (!found)
        {
            unique[count] = arr[i];
            count++;
        }
    }

    printf("\nUnique Elements:\n");

    for (int i = 0; i < count; i++)
        printf("%d ", unique[i]);

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\7.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "7.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 8
Enter array elements:
3 7 3 5 2 5 9 2

Unique Elements:
3 7 5 2 9
```

8. **Bubble Sort**

Program:

```
#include <stdio.h>

int main()
{
   int n;

   printf("Enter array size: ");
   scanf("%d", &n);

   int arr[n];

   printf("Enter array elements:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   for (int i = 0; i < n - 1; i++)
   {
      for (int j = 0; j < n - i - 1; j++)
      {
         if (arr[j] > arr[j + 1])
         {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }

   printf("\nSorted Array:\n");

   for (int i = 0; i < n; i++)
      printf("%d ", arr[i]);

   printf("\n\nTime Complexity: O(n^2)");
```

```
    return 0;
}
```

Output:

```
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "8.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 5
Enter array elements:
64 34 25 12 22

Sorted Array:
12 22 25 34 64

Time Complexity: O(n^2)
```

9. **Binary Search**

Program:

```
#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int binarySearch(int arr[], int n, int key)
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;
```

```
        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int n, key;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);

    printf("Enter key: ");
    scanf("%d", &key);

    int pos = binarySearch(arr, n, key);

    if (pos == -1)
        printf("\nElement %d is not found", key);
    else
        printf("\nElement %d is found at position %d", key, pos + 1);

    printf("\nTime Complexity: O(log n)");

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\9.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "9.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 8
Enter array elements:
3 4 6 -9 10 8 9 30
Enter key: 10

Element 10 is found at position 7
Time Complexity: O(log n)
```

10. **Merge Sort**

Program:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right)
{
   int n1 = mid - left + 1;
   int n2 = right - mid;

   int L[n1], R[n2];

   for (int i = 0; i < n1; i++)
      L[i] = arr[left + i];

   for (int i = 0; i < n2; i++)
      R[i] = arr[mid + 1 + i];

   int i = 0, j = 0, k = left;

   while (i < n1 && j < n2)
   {
      if (L[i] <= R[j])
         arr[k++] = L[i++];
      else
         arr[k++] = R[j++];
   }

   while (i < n1)
      arr[k++] = L[i++];

   while (j < n2)
      arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right)
{
```

```
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main()
{
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("\nSorted Array:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nTime Complexity: O(n log n)");

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\10.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "10.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter array size: 6
Enter array elements:
5 2 9 1 7 3

Sorted Array:
1 2 3 5 7 9
Time Complexity: O(n log n)
```

11. Boundary Paths

Program:

```
#include <stdio.h>

int dp[55][55][55];

int findPaths(int m, int n, int N, int i, int j)
{
   if (i < 0 || i >= m || j < 0 || j >= n)
      return 1;

   if (N == 0)
      return 0;

   if (dp[i][j][N] != -1)
      return dp[i][j][N];

   dp[i][j][N] =
      findPaths(m, n, N - 1, i - 1, j) +
      findPaths(m, n, N - 1, i + 1, j) +
      findPaths(m, n, N - 1, i, j - 1) +
      findPaths(m, n, N - 1, i, j + 1);

   return dp[i][j][N];
}

int main()
{
   int m, n, N, i, j;

   printf("Enter rows: ");
   scanf("%d", &m);

   printf("Enter columns: ");
   scanf("%d", &n);

   printf("Enter steps: ");
   scanf("%d", &N);

   printf("Enter starting row: ");
   scanf("%d", &i);

   printf("Enter starting column: ");
   scanf("%d", &j);

   for (int x = 0; x < 55; x++)
      for (int y = 0; y < 55; y++)
         for (int z = 0; z < 55; z++)
```

```
                dp[x][y][z] = -1;

    printf("\nNumber of Paths = %d", findPaths(m, n, N, i, j));

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\11.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "11.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter rows: 2
Enter columns: 2
Enter steps: 2
Enter starting row: 0
Enter starting column: 0

Number of Paths = 6
```

12. **House Rubber**

Program:

```
#include <stdio.h>

int max(int a, int b)
{
    return (a > b) ? a : b;
}

int robLinear(int arr[], int start, int end)
{
    int prev1 = 0;
    int prev2 = 0;

    for (int i = start; i <= end; i++)
    {
        int temp = max(prev1, prev2 + arr[i]);
        prev2 = prev1;
        prev1 = temp;
    }

    return prev1;
}

int main()
{
    int n;
```

```
    printf("Enter number of houses: ");
    scanf("%d", &n);

    int money[n];

    printf("Enter money in each house:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &money[i]);

    if (n == 1)
    {
        printf("\nMaximum Money = %d", money[0]);
        return 0;
    }

    int ans1 = robLinear(money, 0, n - 2);
    int ans2 = robLinear(money, 1, n - 1);

    printf("\nMaximum Money = %d", max(ans1, ans2));

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\12.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "12.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter number of houses: 3
Enter money in each house:
2 3 2

Maximum Money = 3
```

13. **Climbing Stairs**

Program:

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter number of steps: ");
    scanf("%d", &n);

    if (n == 1)
    {
        printf("\nNumber of Ways = 1");
```

```
    return 0;
}

int first = 1;
int second = 2;

if (n == 2)
{
    printf("\nNumber of Ways = 2");
    return 0;
}

int ways;

for (int i = 3; i <= n; i++)
{
    ways = first + second;
    first = second;
    second = ways;
}

printf("\nNumber of Ways = %d", ways);

return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\13.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "13.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter number of steps: 4

Number of Ways = 5
```

14. **Unique Paths**

Program:

```
#include <stdio.h>

int main()
{
    int m, n;

    printf("Enter number of rows: ");
    scanf("%d", &m);

    printf("Enter number of columns: ");
    scanf("%d", &n);
```

```
    int dp[m][n];

    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i == 0 || j == 0)
                dp[i][j] = 1;
            else
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }

    printf("\nUnique Paths = %d", dp[m - 1][n - 1]);

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\14.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "14.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter number of rows: 7
Enter number of columns: 3

Unique Paths = 28
```

15. **Large Groups**

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char s[100];

    printf("Enter a string: ");
    scanf("%s", s);

    int n = strlen(s);
    int found = 0;

    printf("\nLarge Groups:\n");

    int start = 0;
```

```
    while (start < n)
    {
        int end = start;

        while (end + 1 < n && s[end] == s[end + 1])
            end++;

        if (end - start + 1 >= 3)
        {
            printf("[%d,%d]\n", start, end);
            found = 1;
        }

        start = end + 1;
    }

    if (!found)
        printf("No Large Groups");

    return 0;
}
```

Output:

```
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "15.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter a string: abbxxxxzzy

Large Groups:
[3,6]
```

16. **Game Life**

Program:

```
#include <stdio.h>

int main()
{
    int m, n;

    printf("Enter number of rows: ");
    scanf("%d", &m);

    printf("Enter number of columns: ");
    scanf("%d", &n);

    int board[m][n];
    int next[m][n];
```

```
printf("Enter the board:\n");

for (int i = 0; i < m; i++)
{
   for (int j = 0; j < n; j++)
   {
      scanf("%d", &board[i][j]);
   }
}

for (int i = 0; i < m; i++)
{
   for (int j = 0; j < n; j++)
   {
      int live = 0;

      for (int x = -1; x <= 1; x++)
      {
         for (int y = -1; y <= 1; y++)
         {
            if (x == 0 && y == 0)
               continue;

            int r = i + x;
            int c = j + y;

            if (r >= 0 && r < m && c >= 0 && c < n)
               live += board[r][c];
         }
      }

      if (board[i][j] == 1)
      {
         if (live < 2 || live > 3)
            next[i][j] = 0;
         else
            next[i][j] = 1;
      }
      else
      {
         if (live == 3)
            next[i][j] = 1;
         else
            next[i][j] = 0;
      }
   }
}
```

```
    printf("\nNext State:\n");

    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("%d ", next[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\16.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "16.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter number of rows: 4
Enter number of columns: 3
Enter the board:
0 1 0
0 0 1
1 1 1
0 0 0

Next State:
0 0 0
1 0 1
0 1 1
0 1 0
```

17. **Champagne Tower**

Program:

```
#include <stdio.h>

int main()
{
    int poured, queryRow, queryGlass;

    printf("Enter poured cups: ");
    scanf("%d", &poured);

    printf("Enter query row: ");
    scanf("%d", &queryRow);

    printf("Enter query glass: ");
```

```
    scanf("%d", &queryGlass);

    double tower[101][101] = {0};

    tower[0][0] = poured;

    for (int i = 0; i < 100; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            if (tower[i][j] > 1.0)
            {
                double extra = (tower[i][j] - 1.0) / 2.0;
                tower[i + 1][j] += extra;
                tower[i + 1][j + 1] += extra;
                tower[i][j] = 1.0;
            }
        }
    }

    double ans = tower[queryRow][queryGlass];

    if (ans > 1.0)
        ans = 1.0;

    printf("\nGlass Fill = %.5lf", ans);

    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1\17.exe
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> gcc "17.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 1> .\program.exe
Enter poured cups: 1
Enter query row: 1
Enter query glass: 1

Glass Fill = 0.00000
```